

Сравнение времени и памяти при обработке синтетических данных

Python pandas SQLite

Самостоятельный отчёт по измерениям от 1 октября 2026 года

На одном MacBook Air с Apple M3 и 16 GB RAM сравнивались три конкретные реализации одних и тех же операций. На миллионе строк pandas имел наименьшее медианное время во всех измеренных условиях. Однако на 50 тысячах строк warm-фильтр быстрее выполнил Python, а включение подготовки данных изменило этот порядок. Для JOIN SQLite использовал меньше peak RSS, но выполнялся дольше pandas.

Вопрос и цель

Как меняются время выполнения и пик памяти, когда одинаковые данные обрабатываются циклами Python, средствами pandas и запросами SQLite? Цель состоит в проверяемом сравнении фильтрации, группировки и JOIN с агрегацией, а также в понимании того, как граница таймера влияет на выбор инструмента.

Такой вопрос возникает при разработке локальных аналитических скриптов: один расчёт может выполняться однократно после загрузки, другой многократно на уже подготовленной таблице. Быстрая повторная операция может сочетаться с дорогой подготовкой или большим пиком памяти. Эксперимент показывает эти различия на узком учебном workload; экономия денег, влияние на клиента и производственный SLA не измерялись.

Матрица сравнения

Фактор	Условия
Размер таблицы	50 000 250 000 1 000 000 строк
Методы	Python loops pandas SQLite
Операции	Фильтр Группировка JOIN и агрегация
Режимы и повторы	Cold и warm 5 независимых повторов
Объём измерений	270 основных и 36 отдельных пилотных

270 = 3 размера × 3 метода × 3 операции × 2 режима × 5 повторов. На каждый основной вариант приходится пять новых процессов. Пилот проверял корректность и допустимую нагрузку; его числа не включены в итоговые медианы.

Навигация: [данные и программа](#) · [измерения](#) · [результаты](#) · [проверки](#) · [воспроизведение](#) · [портфолио](#)

Данные и работа программы

Исходный workload полностью synthetic. Таблицы детерминированно сгенерированы NumPy Generator с PCG64 и seed 20261001, а справочник использует отдельный поток seed + 1. Это не готовый чужой benchmark и не данные реальных клиентов. Первичные исследовательские данные появились в результате фактических запусков: elapsed time, peak RSS, порядок, метаданные и проверки ответа.

Поле таблицы фактов	Смысл и диапазон
id	Уникальный возрастающий идентификатор строки
customer_id	Ключ одного из 5 000 клиентов
category_id	Одна из 64 категорий
amount_cents	Целое значение от -1 000 до 10 000 включительно
status	Одно из четырёх значений от 0 до 3

Справочник содержит 5 000 уникальных customer_id и восемь segment_id. Во всех основных входах ключи таблицы фактов имеют соответствие в справочнике. Поля хранятся как int64; текстовых полей, null и дубликатов ключа справочника нет. Наборы разных размеров детерминированны, но не являются гарантированно вложенными выборками.

Операции и общий результат

Фильтр выбирает status = 1 и amount_cents ≥ 5000. Все методы возвращают полный список пар id и amount_cents, упорядоченный по id. На миллионе исходных строк получилось 113 946 строк ответа.

Группировка по category_id возвращает количество строк и сумму amount_cents для каждой категории, с упорядочиванием по ключу. JOIN связывает факты со справочником по customer_id, затем считает строки и суммы по segment_id. Это inner JOIN с последующей агрегацией, а не измерение одного только соединения.

Как устроен код

dataset.py создаёт общие NPZ-файлы и независимые эталонные digest. backends.py содержит три понятные реализации: Python работает со списками строк и словарём клиентов, pandas с DataFrame, SQLite с базой в памяти. worker.py измеряет один запуск; collect.py запускает процессы последовательно и сохраняет журнал. analyze.py считает статистики и строит графики; audit_results.py сверяет сохранённые доказательства.

Все методы читают те же NPZ-файлы. SQL использует первичный ключ справочника, без индексов таблицы фактов; вставка идёт порциями по 10 000 строк. Сравниваются эти реализации и пути подготовки, а не лучшие возможные алгоритмы каждого инструмента.

Что именно измерялось

Режим	Что входит во время	Что остаётся за таймером
Process cold	Чтение NPZ, подготовка backend, операция, сортировка и полный ответ	Запуск процесса, импорт библиотек, проверка digest
Warm	Повторная операция, сортировка и полный ответ	Загрузка, подготовка, один прогрев, удаление его ответа, gc.collect и проверка digest

Cold здесь означает новый процесс, но не холодный диск: кеш файлов ОС не очищался. В warm для каждого повтора тоже создаётся новый процесс, затем выполняется один нетаймируемый прогрев. Все методы материализуют канонические Python-кортежи внутри измеряемой операции. Поэтому результат включает стоимость получения полного ответа, а не только вычислительного ядра.

Время получено через `time.perf_counter_ns`. Основная память измерена `resource.getrusage(RUSAGE_SELF).ru_maxrss` сразу после получения ответа, до вычисления digest. На macOS показатель нормализуется из байтов в MiB, где 1 MiB = 1 048 576 байт. Это исторический максимум RSS всего процесса, включая импорты, исходные массивы, backend и warm-прогрев. Он не равен дополнительной памяти `queue` или уникально принадлежащей процессу физической памяти.

Порядок и ограничения нагрузки

Порядок блоков размер × повтор и порядок метода, операции и режима внутри блока случайны с сохранённым seed 73129; у пилота seed 73130. Измерительные процессы не запускались одновременно. Между блоками была пауза в одну секунду. Проверялись эквивалентность, таймаут, peak RSS и системный счётчик `swaptouts`.

Пилот на 10 000 и 50 000 строках завершился успешно, после него исходная матрица была сохранена без изменений. Лимиты: 900 с суммарного wall time процессов, 1 200 с на кампанию, 30 с на процесс и 2 GiB RSS после завершения worker. Фактически процессы заняли 90,20 с вместе с пилотом; максимум RSS равен 282,08 MiB. Остановок по ограничениям не было.

Проверенная среда

Компонент	Зафиксированное значение
Компьютер	MacBook Air Mac15,12 Apple M3 16 GB RAM
CPU и ОС	8 ядер 4 performance и 4 efficiency macOS 26.7.1
Python и библиотеки	Python 3.12.14 NumPy 2.3.5 pandas 2.2.3
SQLite	3.53.1 :memory: temp_store=MEMORY threads=1

OMP, OpenBLAS, MKL, VECLIB и NumExpr получили лимиты потоков 1. SQLite threads=1 допускает до одного вспомогательного потока сверх вызывающего; строго однопоточным сравнение не названо. Закрепления за ядрами не было. [Источник определения: официальная документация SQLite.]

Результаты по времени

В таблице показаны медиана и диапазон пяти повторов на миллионе строк. Единица измерения: миллисекунды. В каждой ячейке первая строка содержит медиану, вторая наблюдаемые минимум и максимум. Результаты полностью получены из сохранённого raw.json.

Операция и режим	Python	pandas	SQLite
Фильтр Cold	223,09 214,32-241,60	27,94 27,37-28,50	509,94 506,12-521,71
Группировка Cold	173,93 170,22-176,43	17,51 16,93-18,39	815,28 797,98-841,72
JOIN и агрегация Cold	204,47 203,02-212,42	45,89 44,42-51,94	767,63 766,16-768,71
Фильтр Warm	27,55 26,63-28,12	17,68 17,04-18,13	64,07 63,57-64,65
Группировка Warm	72,13 71,67-72,88	6,62 6,51-7,38	380,16 348,19-433,14
JOIN и агрегация Warm	107,68 106,19-108,11	28,05 26,99-29,67	317,70 312,57-453,98

pandas имел наименьшие медианы во всех шести условиях на миллионе строк. Отношения медиан Python / pandas в warm равны 1,56 для фильтра, 10,89 для группировки и 3,84 для JOIN. Например, $72,134791 / 6,624583 \approx 10,89$. Отношение относится к совпадающему размеру, операции и режиму; это не среднее отношение отдельных случайных запусков.

Почему важен маленький набор

Фильтр на 50 000 строк	Python	pandas	SQLite
Cold	9,94 9,61-11,47	3,42 3,16-3,50	29,74 29,23-30,02
Warm	1,13 1,12-1,22	1,35 1,24-1,39	3,10 3,02-3,20

Warm-медиана Python составляет 1,13 мс против 1,35 мс у pandas. С загрузкой и подготовкой pandas быстрее: 3,42 против 9,94 мс. Среди девяти пар размер × операция это единственное изменение порядка медиан между cold и warm. Разница warm-медиан около 0,23 мс; её нельзя превращать в обещание существенного бизнес-эффекта.

Преимущество циклов на малом фильтре совместимо с влиянием постоянных накладных расходов, а рост преимущества pandas при группировке совместим с эффективной реализацией табличных операций. Это интерпретации: эксперимент не выделял отдельными замерами стоимость каждого механизма и не устанавливал такую причинность.

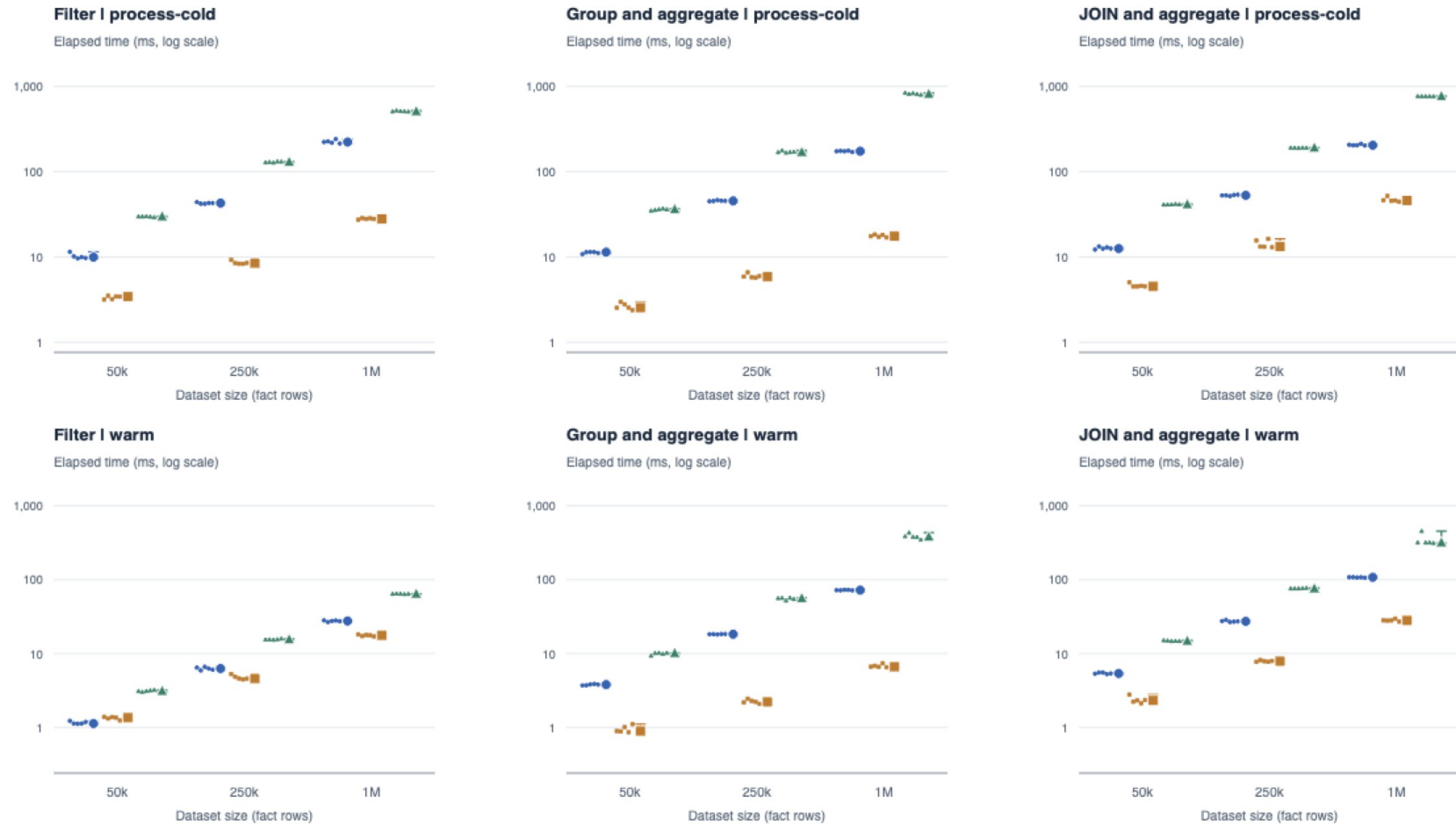
На следующей странице показаны все 270 основных наблюдений. Размеры на оси X расположены как категории, ось времени логарифмическая. Шкалы одинаковы внутри одного режима, но различаются между cold и warm. Малые метки обозначают повторы, большие медианы, вертикальные отрезки минимум и максимум.

Elapsed time

Synthetic integer workload | One machine | Apple M3 MacBook Air, 16 GB RAM

5 independent repeats per cell | 270 measured runs | Same source data and exact result checks

● Python loops ■ pandas ▲ SQLite Small marks: individual runs Large marks: median Bars: observed min-max



Bars show the observed minimum and maximum, not confidence intervals. Dataset sizes are equally spaced categories.

Time y-axis limits differ between cold and warm rows; they are shared across operations within each mode.

Cold time includes loading, backend preparation and query output. Warm time includes only the query and its complete output.

RSS is the peak of the whole worker through query completion: Imports, source data, backend setup and warm priming included.

Process-cold is not disk-cold; OS file cache was not cleared. One machine, one synthetic schema. No energy or CO2 measurement.

Результаты по памяти

Сравнение памяти следует читать отдельно от сравнения времени. Ниже показан peak RSS всего warm-worker на миллионе строк: медиана и наблюдаемый диапазон, MiB. Импорты и подготовка не входят в warm-таймер, но входят в этот показатель памяти.

Операция	Python	pandas	SQLite
Фильтр	281,78 281,75-281,81	142,84 142,78-142,91	113,88 113,86-113,91
Группировка	274,73 274,64-274,75	123,12 123,03-123,25	128,92 128,89-129,25
JOIN и агрегация	274,69 274,64-274,81	229,83 229,77-229,86	128,91 128,83-128,94

Для JOIN SQLite достигает медианы 128,91 MiB против 229,83 MiB у pandas. В том же условии pandas выполняет операцию за 28,05 мс, SQLite за 317,70 мс. Поэтому вариант с меньшим RSS не оказался вариантом с меньшим временем. У Python медианный RSS этого JOIN около 274,69 MiB.

Python сохраняет данные как обычные списки и целочисленные объекты; pandas использует int64 DataFrame поверх массивов, а SQLite держит собственную базу. Эти представления и наборы импортируемых библиотек различаются. Измеренный RSS описывает процесс целиком и не позволяет приписать каждое дополнительное MiB одному алгоритму.

На 50 тысячах строк библиотечные и другие постоянные затраты занимают заметную часть процесса, поэтому порядок по памяти может отличаться от порядка на миллионе строк. Peak RSS после прогрева остаётся историческим максимумом, даже если временные структуры уже освобождены. Из малого разброса RSS нельзя делать вывод об одинаковых внутренних траекториях потребления памяти.

Что произошло с исходными гипотезами

H1 ожидала преимущества pandas над простыми циклами при warm-фильтрации и группировке на больших данных. На миллионе строк ожидание поддержано описательно; маленький фильтр показывает границу такого обобщения.

H2 предполагала возможное различие времени и памяти для SQL JOIN и pandas merge. Это было исследовательское ожидание без заданного направления. В измеренном JOIN SQLite потребовал меньше RSS, а pandas меньше времени.

H3 ожидала, что включение подготовки может менять порядок методов. Наблюдаемое изменение произошло для фильтра на 50 тысячах строк. В остальных восьми сочетаниях размера и операции порядок медиан не менялся. Статистические критерии значимости не применялись.

Следующая страница показывает память по всем размерам, операциям и режимам. Шкала начинается с нуля и едина для всех шести панелей. Графики времени и памяти используют одни и те же исходные измерения, но разные границы учёта затрат.

Peak whole-worker memory

Synthetic integer workload | One machine | Apple M3 MacBook Air, 16 GB RAM

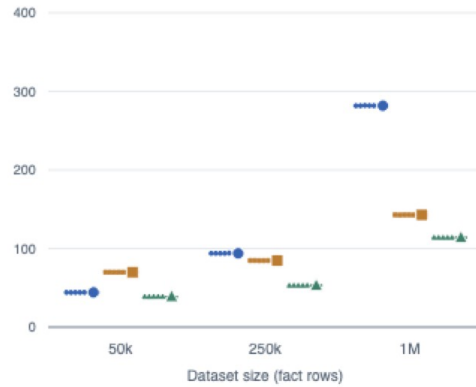
5 independent repeats per cell | 270 measured runs | Same source data and exact result checks

● Python loops ■ pandas ▲ SQLite

Small marks: individual runs Large marks: median Bars: observed min-max

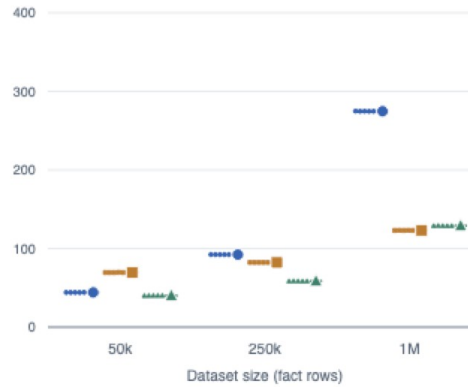
Filter | process-cold

Peak RSS (MiB, zero baseline)



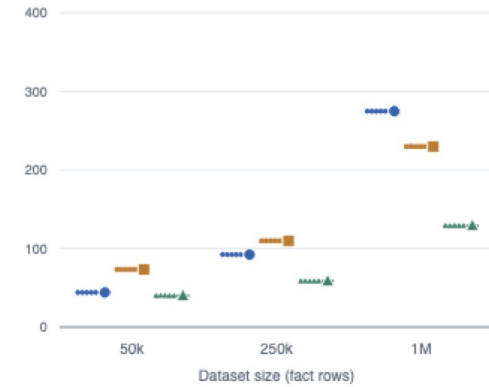
Group and aggregate | process-cold

Peak RSS (MiB, zero baseline)



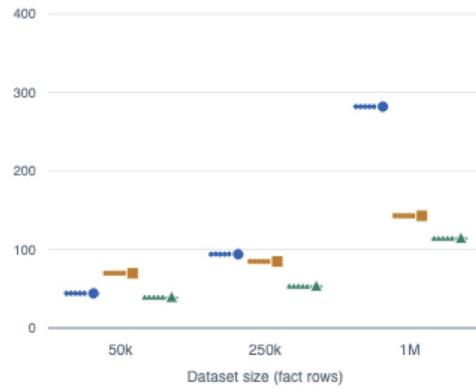
JOIN and aggregate | process-cold

Peak RSS (MiB, zero baseline)



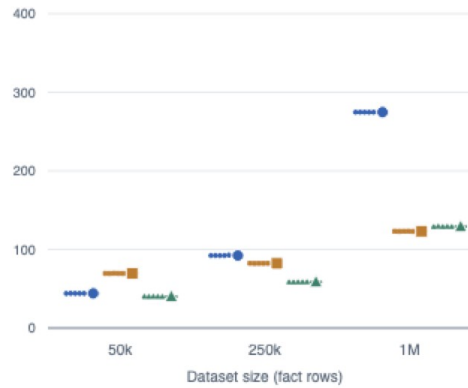
Filter | warm

Peak RSS (MiB, zero baseline)



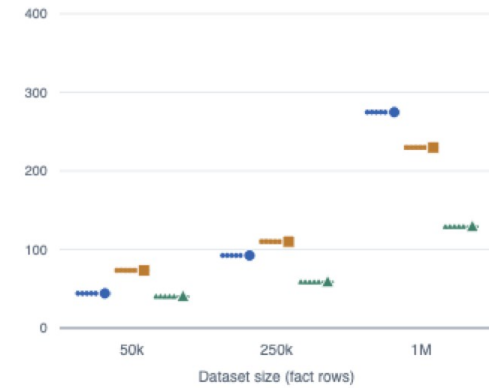
Group and aggregate | warm

Peak RSS (MiB, zero baseline)



JOIN and aggregate | warm

Peak RSS (MiB, zero baseline)



Bars show the observed minimum and maximum, not confidence intervals. Dataset sizes are equally spaced categories.

Memory uses the same zero-based y-axis across all six panels.

Cold time includes loading, backend preparation and query output. Warm time includes only the query and its complete output.

RSS is the peak of the whole worker through query completion: Imports, source data, backend setup and warm priming included.

Process-cold is not disk-cold; OS file cache was not cleared. One machine, one synthetic schema. No energy or CO2 measurement.

Проверки и границы выводов

Перед измерениями были успешно выполнены шесть тестовых методов. Независимые маленькие примеры имеют вручную рассчитанные ожидаемые ответы; они проверяют все три backend и отдельный NumPy-эталон. Проверка не сводится к согласию двух похожих реализаций.

1. Порог, отрицательные значения и отсутствующий клиент: значение 4999 исключается, 5000 включается при нужном status; сумма и inner JOIN проверяются на известных ответах.
2. Пустой результат фильтра и отсутствующие категории: проверяется пустой список и отсутствие искусственно добавленных групп.
3. Пустая таблица фактов: все три операции должны возвращать пустой результат.
4. Пустой справочник: JOIN пуст, хотя фильтр и группировка фактов могут иметь строки.
5. Детерминированность и сохранение итогов: повторная генерация даёт те же массивы, ключи уникальны, JOIN и группировка сохраняют количество и сумму основных данных.
6. Полнота и воспроизводимость расписания: одинаковый seed даёт одинаковый порядок, другой seed меняет порядок, все комбинации присутствуют без дублей.

Для каждого из 306 измеренных ответов отдельно проверялись число строк и SHA-256 канонических целочисленных кортежей. Контрольные суммы исходных файлов, методики и измерительного кода сохранены. У 270 основных прогонов разные PID. После сбора независимо сверены 540 сводных статистик в 54 условиях, полнота расписания и совпадение JSON, JSONL и CSV.

Разброс и внешняя применимость

Пять повторов показывают наблюдаемую изменчивость, но не дают надёжного универсального распределения производительности. Минутный loadavg системы изменялся от 4,17 до 5,24 на восьми ядрах; он включает эксперимент и другие процессы. Фоновые приложения, привязка к ядрам и температурный режим не контролировались. Рандомизация уменьшает зависимость от порядка, но не устраняет эти факторы.

Например, warm-JOIN SQLite на миллионе строк лежал в диапазоне 312,57-453,98 мс при медиане 317,70 мс. Максимум сохранён в данных и на графике. Диапазоны не названы доверительными интервалами; выбросы не удалялись ради более красивого результата.

В исследовании нет текстовых полей, null, намеренно перекошенных распределений, many-to-many JOIN, дисковой базы, оптимизации индексов или других машин. SQL-подготовка оплачивается заново в cold; для уже существующей базы такая граница может быть неуместна. Температура и энергия не измерялись. powermetrics требовал повышенных прав, поэтому из времени и памяти не выводятся энергия, CO2 или экологическая польза.

Воспроизводимый запуск и материалы

Для проверки выводов достаточно сохранённых raw-данных: новый benchmark не требуется. При подготовке этой расширенной версии измерительные запуски не повторялись. Используйте Python 3.12 и версии из requirements.txt; для DOCX дополнительно нужен python-docx, для его PDF-рендера LibreOffice и Poppler из поддерживаемой среды.

Пересчёт статистики без новых измерений

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python src/analyze.py --machine-label "Apple M3 MacBook Air, 16 GB RAM"
python src/audit_results.py
```

Команды запускаются из корня проекта. Название машины относится к включённым измерениям и должно быть изменено при анализе другого запуска. На macOS PNG экспортируется через sips, на поддерживаемой среде также возможен pdftorpm. Числа должны воспроизводиться из raw; новое измерение времени на другой машине совпадать не обязано.

Если позже нужен собственный повтор

Создайте отдельную пустую копию с src, tests, docs, requirements.txt и файлами config. Сохраните исходные результаты. Сначала выполните тесты и пилот, проверьте его корректность и нагрузку, затем используйте замороженную матрицу только если она остаётся допустимой. Это инструкция для будущего запуска, а не выполненная в этой версии работа.

```
mkdir -p results
python -m unittest discover -s tests -v
python src/dataset.py --sizes 10000 50000 250000 1000000
python src/collect.py --phase pilot
python src/collect.py --phase main --design config.main.json
```

Сборщик запрещает перезапись готовой коллекции и останавливается при нарушении проверок или лимитов. Неудачные и частичные результаты сохраняются; анализатор не принимает неполную основную матрицу.

Что находится в проекте

[Первичные CSV и JSON](#). results/main и results/pilot: каждый замер, журнал JSONL, порядок и метаданные.

[Полные статистики](#). Медианы, квартили, минимум и максимум; ratios.csv содержит явные базовые отношения.

[Методика до сбора](#). Исходный протокол, IMPLEMENTATION_ADDENDUM.md и решение после пилота.

[Архив исследования](#). 57 файлов и MANIFEST.json: исходные NPZ, код, тесты, графики, README EN и IT, краткий отчёт.

Ссылки на соседние файлы рассчитаны на документ в папке reports. При отправке отдельного PDF приложите архив исследования. Данная расширенная версия сохранена отдельно в PDF и DOCX; прежний краткий отчёт и архив не заменены.

Определение SQLite threads: [официальная документация PRAGMA threads](#)

Как это усиливает портфолио

Проект показывает путь от исследовательского вопроса до проверяемого вывода. Его ценность для junior-портфолио проявляется, когда вы можете объяснить решения, найти исходное наблюдение и воспроизвести расчёт.

Роль и вклад

Пользователь выбрал тему и задал требования к методике, безопасности, коду и материалам. Код, измерения и оформление выполнены в этой сессии Codex. В личном описании отдельно называйте проверки и изменения, которые вы действительно выполнили; независимое авторство всех компонентов пользователю не приписывается.

Какие навыки можно показать

Экспериментальный дизайн. Объяснить гипотезы, фиксированную матрицу, пилот, seed, рандомизацию, критерии остановки и отличие повторов от независимых машин.

Python и SQL. Прочитать три реализации, обосновать фильтр и группировки, уникальность ключа JOIN и общий формат результата.

Профилирование. Нарисовать границы cold и warm, объяснить perf_counter_ns и почему peak RSS не является памятью одной операции.

Проверка результатов. Проследить граничный пример через каждый backend, объяснить NumPy-эталон, digest и проверки сохранения количества и суммы.

Анализ и визуализация. Пересчитать медиану и отношение по raw, прочитать логарифмическую ось и отделить наблюдаемый диапазон от доверительного интервала.

Честные формулировки для CV

Эти варианты описывают содержание проекта без заявления о ручном или независимом авторстве. Используйте их, когда можете подтвердить указанные навыки демонстрацией.

Reproducible performance study of Python, pandas and SQLite on synthetic integer data, covering 270 main trials, process-cold and warm timing, peak worker RSS, and exact-result validation.

Portfolio analysis of time and memory tradeoffs in filtering, aggregation and joins, with deterministic inputs, six correctness tests, randomized execution order, raw measurement logs and charts showing observed variability.

Короткое описание для LinkedIn

This portfolio study compares three implementations on identical synthetic data. At one million rows, pandas had the lowest median elapsed time in the tested workloads; a smaller warm filter favored Python, while SQLite used less peak RSS for the large JOIN. The repository includes the protocol, raw measurements, validation and reproducible analysis. These findings describe one machine and one schema, not a universal ranking.

Подготовка к собеседованию

Перед обсуждением откройте `backends.py`, одну строку `raw.csv` и соответствующую сводную строку. Подготовьте короткое объяснение и небольшой фрагмент кода для каждого вопроса. Не требуется заново запускать всю матрицу ради собеседования.

Почему `cold` и `warm` дают разные результаты

Покажите границу таймера в `worker.py`. `Cold` включает NPZ и подготовку, `warm` начинается после прогрева и освобождения его ответа. Приведите фильтр на 50 тысячах строк: Python 1,13 мс против pandas 1,35 мс в `warm`; 9,94 против 3,42 мс в `cold`. Уточните, что `cold` не означает очищенный кеш ОС.

Как доказать что три метода решают одну задачу

Разберите fixture с 4999, 5000, отрицательной суммой и отсутствующим клиентом. Объясните условия `inner JOIN`, одинаковые сортировку и материализацию, эталонные целочисленные ответы и совпадение `digest`. Скажите, что проверка на одном примере не заменяет проверку каждого основного ответа.

Что именно означает меньший `peak RSS` у `SQLite`

Сравните большой `warm-JOIN`: 128,91 MiB у `SQLite` и 229,83 MiB у `pandas`. Объясните, что `RSS` включает библиотеку, входы, подготовку и прогрев; это максимум всего процесса, а не объём выделений одной `query`. Меньшая память в этом случае сопровождалась большим временем.

Можно ли считать `pandas` всегда более быстрым

Ответьте через область применимости: одна машина, `int64` без `null` и текстовых полей, выбранные реализации и индексы. Покажите малый `warm-фильтр` как контрпример универсальному утверждению. При переносе на рабочую задачу сначала уточняются реальные типы, распределение, формат хранения и то, включается ли подготовка.

Как проверить отношение времени Python и pandas

Отфильтруйте `raw` по 1 000 000 строк, `group` и `warm` отдельно для Python и pandas. Возьмите медианы пяти `elapsed_seconds` и поделите Python на pandas. Объясните единицы, сохранение всех повторов, диапазон вместо доверительного интервала и то, что 10,89 раза быстрее не означает такой же экономии стоимости или энергии.

Чтобы личный вклад стал сильнее, выполните небольшой проверяемый шаг самостоятельно: объясните один тест, пересчитайте одну медиану и исправьте одну понятную часть документации. Зафиксируйте фактическое изменение и результат проверки. Новые заявления о навыках должны следовать за выполненной работой, а не появляться заранее.